

CASHALL WEBSITE → PARTNER APP TECHNICAL HANDOFF

End-to-End System Architecture, Supabase Database Specification & Unified Mobile Integration Guide

Target Audience:	Partner & Delivery Mobile App Engineering Team	Target Date:	August 2026
Core Backend:	Next.js 14 Serverless on Vercel (TypeScript)	Database:	Supabase PostgreSQL (Project jqysknhobtpcbyylnfc)
Scope:	Read-Only Inspection, Flow Documentation & API Contracts	Status:	Production Verified & Operational

1. Executive Summary

CashALL operates a specialized device re-commerce buyback service across West Bengal and Uttar Pradesh, buying pre-owned smartphones, laptops, and tablets directly from consumers. The customer workflow delivers an online estimated quote, captures doorstep pickup details, schedules a local appointment, and dispatches a CashALL field logistics partner to physically inspect the hardware, verify seller identity (KYC), capture digital signatures, disburse instant payout funds (via UPI), and issue an authorized bill of sale. The web application and the mobile partner app share **one unified backend infrastructure** powered by Supabase. There are no separate databases, no duplicate tables, and no fragmented business logic. Both platforms operate as two synchronized clients interacting with the same PostgreSQL database.

2. Current System Architecture

- **Client Tier:** Web application built with Next.js 14 App Router, Tailwind CSS, and Framer Motion. Mobile Partner App built with Flutter or React Native using official Supabase client libraries.
- **API & Application Tier:** Next.js Serverless API endpoints deployed on Vercel acting as the trusted server environment. Manages role authorization (CUSTOMER, AGENT, PARTNER, ADMIN), state transitions, SHA-256 PII hashing for Meta CAPI, and privileged database operations.
- **Database Tier:** Hosted PostgreSQL database managed by Supabase in the aws-0-ap-south-1 (Mumbai, India) AWS region. Queried by the backend through Prisma ORM using transaction-mode connection pooling (PgBouncer IPv4).
- **Storage Tier:** Supabase Storage Buckets storing device photos, inspection evidence, and payment receipts.

3. Supabase Project Structure & Configuration

Parameter	Environment Variable Name	Client Exposure (Mobile App)	Description
Project URL	NEXT_PUBLIC_SUPABASE_URL	YES (Embed in App)	https://jqysknhobtpcbyylnfc.supabase.co
Anon / Public Key	NEXT_PUBLIC_SUPABASE_ANON_KEY	YES (Embed in App)	JWT with 'anon' role for client-side queries
Publishable Key	NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	YES (Embed in App)	sb_publishable_... client identifier
Service Role Key	SUPABASE_SERVICE_ROLE_KEY	STRICTLY FORBIDDEN	Privileged key bypassing RLS. Server-side only.
Database URL	DATABASE_URL / POSTGRES_PRISMA_URL	STRICTLY FORBIDDEN	PostgreSQL connection string with pooler port 6543.

4. Database Tables Inventory (Audit of 41 PostgreSQL Tables)

A rigorous schema inspection confirms **41 tables** currently active in the database. Below is the breakdown of all core entities relevant to the Partner App:

Intended Entity	Actual Table Name in DB	Schema Status	Architectural Role & Description
brands	Brand	EXISTS	Manufacturer directory (Apple, Samsung, Xiaomi, HP, Dell, etc.)

device_models	DeviceModel	EXISTS	Specific models, release years, category tags, and base prices
model_variants	DeviceVariant	EXISTS	Implemented as DeviceVariant. Links modelId with storage/ram.
storage_variants	Column storage	Not separate table	Expected table not found. Storage is stored on DeviceVariant.storage.
color_variants	ColorVariant	EXISTS	Model/variant color choices, hex codes, and color image assets
question_sets	QuestionSet	EXISTS	Category-level grouping for condition questionnaires
questions	ConditionQuestion	EXISTS	Implemented as ConditionQuestion. Groups: BASIC, SCREEN, BODY, etc.
question_options	ConditionOption	EXISTS	Implemented as ConditionOption. Selectable answers per question.
pricing_rules	PricingRule	EXISTS	Mathematical deduction / bonus rules (% or fixed amounts)
pricing_history	PricingHistory	EXISTS	Audit log of pricing rule changes and author IDs
device_images	DeviceImage	EXISTS	Photo gallery references attached to models

Core Transactional Tables Summary:

Table Name	Primary Key	Foreign Keys	Key Columns & Responsibility
User	id (UUID)	None	phone (unique), name, email, role (CUSTOMER, AGENT, PARTNER, ADMIN), firebaseUid.
Address	id (UUID)	userId → User	house, street, area, landmark, city, state, pincode, lat/lng.
Quote	id (UUID)	variantId → DeviceVariant	quoteNumber (unique), selectedAnswersJson, basePrice, totalDeductions, estimatedPrice, breakdownJson.
Order	id (UUID)	quoteId, userId, addressId, agentId	orderNumber (unique e.g. #CA61962), pickupDate, pickupTimeSlot, status, finalPrice, urn, paymentScreenshotUrl.
Pickup	id (UUID)	orderId → Order, partnerId → Partner	date, timeSlot, status (SCHEDULED, ASSIGNED), notes (agent name), assignedAt.
QcReport	id (UUID)	orderId → Order	inspectorName, imeiNumber, declaredAnswersJson, physicalAnswersJson, revisedPrice, photoUrlsJson.
Offer	id (UUID)	orderId → Order	originalPrice, revisedPrice, priceDifferenceReason, status (PENDING, ACCEPTED).
Payment	id (UUID)	orderId → Order	amount, method (UPI), status (PAID), transactionRef (Bank UTR), paidAt.
Imei	id (UUID)	orderId → Order (optional)	code (unique 15-digit), status (VERIFIED), checkedAt, comments.
ImageReference	id (UUID)	entityId → Order	url (signed), bucket (qc-reports, etc.), key (storage path), entityType.

5. Database Relationships & Entity Relationship Model

- **Brand 1:N DeviceModel:** Deleting a brand cascades to its models.
- **DeviceModel 1:N DeviceVariant:** Variants define storage options (128GB, 256GB) and base prices.
- **DeviceVariant 1:N Quote:** Quotes store the snapshot of answers and pricing at calculation time.
- **Quote 1:1 Order:** An order converts a quote into a scheduled pickup commitment.
- **User 1:N Order:** Both customer (userId) and internal agent (agentId) link to the User table.
- **Order 1:N Pickup:** Stores appointment date, time window, and partner assignment.
- **Order 1:N QcReport & Offer:** Captures field inspection results and revised price negotiations.
- **Order 1:N Payment & ImageReference:** Connects transaction UTR and Supabase Storage photo paths.

6. Customer Journey — Step-by-Step Flow

Step	Customer Action	Implementation Status	Code Location / Route	Database Data Stored
1	Select Location & Category	IMPLEMENTED	LocationModal.tsx	Browser session / cookies
2	Select Brand & Model	IMPLEMENTED	app/sell/[category]/...	Queried from Brand & DeviceModel
3	Select Storage Variant	IMPLEMENTED	app/calculator/[slug]	Reads DeviceVariant.basePrice
4	Answer Diagnostic Questions	IMPLEMENTED	app/calculator/[slug]	In-memory questionnaire state
5	Generate Valuation Quote	IMPLEMENTED	POST /api/v1/quotes/calculate	Quote table (quoteNumber, estimatedPrice)
6	Customer Phone Auth	IMPLEMENTED	app/checkout/pickup	User table (phone, firebaseUid)
7	Address & Date Selection	IMPLEMENTED	app/checkout/pickup	Address table (pincode, house, city)
8	Confirm Doorstep Pickup	IMPLEMENTED	POST /api/v1/orders/create	Order (#CAxxxxx) & Pickup (SCHEDULED)
9	Admin Assigns Field Partner	IMPLEMENTED	/admin/orders assign route	Order.agentId, status: PARTNER_ASSIGNED
10	Partner Visits & Starts QC	IMPLEMENTED	POST /api/v1/orders/[id]/start-inspection	Order.status: INSPECTION_STARTED
11	Physical Inspection & IMEI	IMPLEMENTED	POST /api/v1/agent/orders/[id]/inspection	QcReport table, Imei table, finalPrice
12	Seller ID Verification (KYC)	PARTIALLY IMPLEMENTED	POST /api/v1/orders/[id]/verify-identity	Masked ID (XXXX-XXXX-1234), ref ID
13	Customer E-Signature	PARTIALLY IMPLEMENTED	POST /api/v1/orders/[id]/esign	SHA-256 agreement hash, declaration
14	Payout via UPI / Bank	IMPLEMENTED	POST /api/v1/agent/upload-payment	Payment (PAID), Order.urn (UTR), screenshot
15	Order Marked Completed	IMPLEMENTED	POST /api/v1/agent/orders/[id]/complete	Order.status: COMPLETED
16	Automated Bill Issued	IMPLEMENTED	GET /api/v1/orders/[id]/generate-bill	Official invoice with GSTIN & IMEI emailed

7. Pricing Engine & Dynamic Valuation

Does the current system support percentage-based deductions per question? YES.
Implemented in lib/services/pricing.service.ts. The valuation pipeline evaluates rules in PricingRule:

- Base Price:** Queried from DeviceVariant.basePrice.
- Deduction Rules:** Each answer matches a rule with adjustmentType:
 - PERCENTAGE_DEDUCTION: deduction = (basePrice * abs(adjustmentValue)) / 100
 - FIXED_DEDUCTION: deduction = abs(adjustmentValue)
 - FIXED_BONUS: Adds value for original charger/box.
- Floor Price Protection:** Guaranteed minimum buyout price clamped to: Math.max(500, Math.round(basePrice * 0.15)).
- Dynamic Administration:** Admins can update rule percentages in PostgreSQL without deploying new code.

8. Supabase Storage Architecture

CashALL utilizes 4 private/signed Supabase Storage buckets configured in StorageService (lib/services/storage.service.ts):

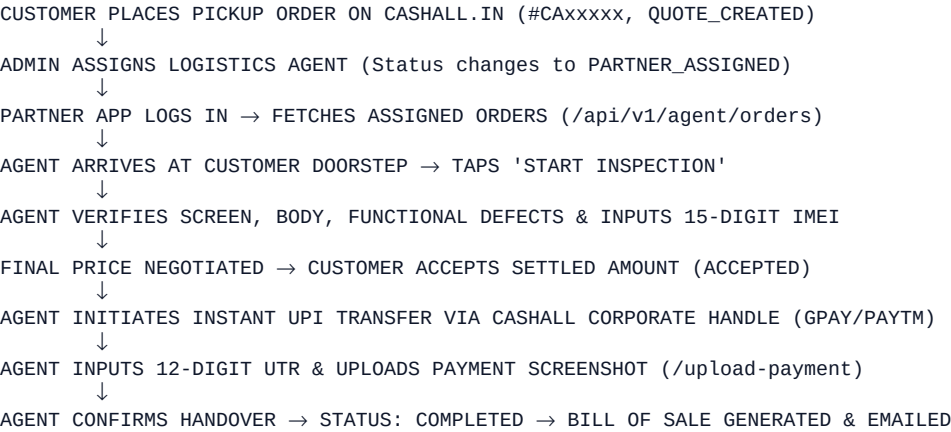
- customer-devices:** Photos uploaded by customers. Path: {userId}/{uuid}.{ext}.
- qc-reports:** Hardware inspection photos uploaded by field agents. Path: {agentId}/{orderId}_{uuid}.{ext}.
- payment-screenshots:** Proof of successful UPI transfer screenshots showing Bank UTR. Path: urn_{orderId}_{timestamp}.{ext}.
- payment-proofs:** Back-office payment receipts.

Access is governed by signed URLs with configurable expiry (3,600s for display, up to 1 year for audit references).

9. Existing APIs Contract for Partner App Integration

Method & Endpoint	Authentication	Input Body / Query	Output & Database Impact
POST /api/v1/agent/login	Public / Agent	{ name, password, phone }	Returns { token, agent: { id, name, role } }.
GET /api/v1/agent/orders	Bearer Token	?agentId={id}&={phone}	Returns array of assigned pickups with address, slot, amount.
GET /api/v1/orders/{id}	Bearer Token	Order ID or #CA number in path	Full order object with quote, address, customer details.
POST /api/v1/orders/{id}/start-inspection	Agent Token	None	Transitions Order to INSPECTION_STARTED.
POST /api/v1/agent/orders/{id}/inspection	Agent Token	{ imei, screenFinding, bodyFinding, revisedPrice, reason, agentName }	Creates QcReport, updates Imei, locks finalPrice.
POST /api/v1/agent/upload-payment	Agent Token	Multipart: orderId, urn, file	Uploads screenshot to Supabase Storage, sets Order.urn.
POST /api/v1/agent/orders/{id}/complete	Agent Token	{ finalPrice, utr, agentName }	Sets Payment PAID, Order COMPLETED, dispatches bill email.
GET /api/v1/orders/{id}/generate-bill	Public / Agent	Order ID or #CA number in path	Returns structured bill JSON with GSTIN, IMEI, payout.

10. Partner App Operational Data Flow



11. Security & Row Level Security (RLS) Rules

Zero-Trust Client Boundary: The Partner App must execute with **Public/Anon credentials only**. The SUPABASE_SERVICE_ROLE_KEY and direct database credentials (postgresql://postgres...) must **NEVER** be compiled into mobile binaries (APK/IPA). If decompiled, an embedded service role key compromises the entire database.

- **Public Read Access:** Brand, DeviceModel, DeviceVariant, ConditionQuestion, ConditionOption, ServiceArea.
- **Agent Scoped Access:** Orders are scoped strictly to the authenticated agent's assigned ID. Agents cannot read or modify unassigned orders.
- **Audit Trail:** Every state change, inspection write, and price adjustment writes an immutable record to AuditLog.

WHAT THE APP DEVELOPER NEEDS FROM CASHALL

Category	Items & Architectural Details
A. Credentials & Config	1. Supabase Project URL: https://jqysknhobtpcbpyltnfc.supabase.co 2. Supabase Anon Key: JWT string with 'anon' role (embeddable in app). 3. Supabase Publishable Key: sb_publishable_P-MbOk1yAxihXTOFUxt54w_o_hpl3fl 4. CashALL API Base URL: https://www.cashall.in/api/v1

B. APIs to Call	• POST /api/v1/agent/login (Agent authentication) • GET /api/v1/agent/orders (Assigned pickups list) • POST /api/v1/orders/[id]/start-inspection (Status transition) • POST /api/v1/agent/orders/[id]/inspection (Submit QC & IMEI) • POST /api/v1/agent/upload-payment (Upload UTR & screenshot) • POST /api/v1/agent/orders/[id]/complete (Mark finished) • GET /api/v1/orders/[id]/generate-bill (Fetch receipt)
C. Supabase Tables	• Direct Reads: Brand, DeviceModel, DeviceVariant, ConditionQuestion, ConditionOption. • Indirect Reads/Writes (via API): Order, Address, QcReport, Imei, Payment, ImageReference.
D. Permissions Needed	• Permission to read assigned orders for logged-in agent. • Permission to upload inspection and payment photos to Supabase Storage. • Permission to transition order statuses through the state machine.
E. Prohibited Access	• NEVER access SUPABASE_SERVICE_ROLE_KEY or direct DB strings. • Must NOT access other agents' unassigned orders. • Must NOT alter base buyout prices or system pricing rules.
F. To Be Built in App	1. Native mobile UI (Flutter / React Native) with offline caching. 2. Camera integration with Barcode / IMEI scanner for box/SIM tray scanning. 3. Touchpad signature canvas for customer digital declaration. 4. UPI intent launcher (e.g. upi://pay?pa=...&am;=...) for direct payment.